

Integration Architecture Discussion (Polyglot, Docker-based)

Scaling to 10+ Systems – Event-Driven Architecture

Based on the current code base, scaling beyond a few systems requires removing tight coupling between integrations. The existing design already establishes this foundation by using Kafka as an event backbone.

As the number of systems grows, polling is replaced or augmented with event-driven mechanisms such as webhooks. Each external system publishes domain events into Kafka topics. Kafka provides buffering, replayability, and back-pressure handling, allowing producers and consumers to scale independently.

Bulk Processing, API Gateways, and Circuit Breakers

Bulk APIs reduce the number of external calls by fetching or exporting data in batches. An API Gateway centralizes authentication, caching, and rate limiting. Circuit breakers prevent cascading failures by temporarily stopping calls to failing systems.

Integration Platforms (NiFi / MuleSoft)

When integrations grow significantly, platforms like Apache NiFi or MuleSoft can replace custom adapters. They handle routing, transformation, and protocol translation while Kafka remains the central event bus.

Polyglot Pipeline (Java + Python via Docker)

The polyglot architecture is achieved through Docker rather than language-level coupling. Spring Boot producers and Python consumers run in separate containers orchestrated via Docker Compose. Kafka acts as the language-agnostic contract between services.

End-to-End Pipeline Flow

1. External systems expose data via REST, SOAP, or batch APIs.
2. Java Spring Boot producers fetch, cache, and publish events to Kafka.
3. Kafka buffers and distributes events.
4. Python consumers process events asynchronously.
5. Redis ensures idempotency.
6. Analytics service receives processed results.

Adapting to a Java-Only Stack

The consumer could be rewritten using Spring Kafka with async processing via thread pools or reactive programming. Redis-based idempotency and retry logic remain unchanged. The architecture is language-agnostic.

Async Processing, Retries, and Idempotency

Async processing enables high throughput. Retry mechanisms handle transient failures. Redis-based idempotency guarantees exactly-once business outcomes.